EXPERIMENT 5: Prompt Engineering (Zero-shot, Few-shot, CoT)

Aim

To analyze the impact of Zero-shot, Few-shot, and Chain-of-Thought prompting on the reasoning capabilities of Large Language Models.

Algorithm

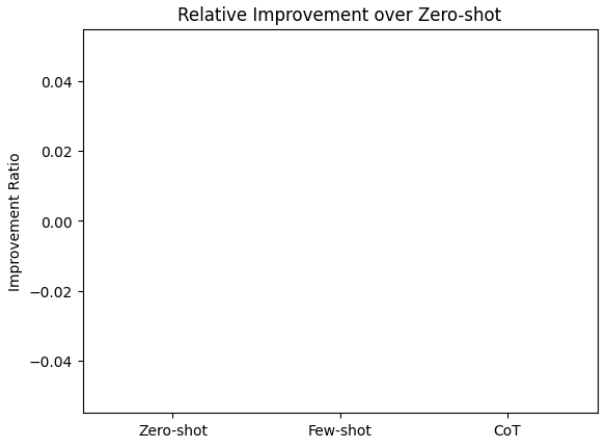
1. Load GSM8K dataset
2. Initialize text generation model
3. Apply Zero-shot, Few-shot, and CoT prompting
4. Generate responses
5. Compare with actual answers
6. Compute accuracy
7. Visualize performance

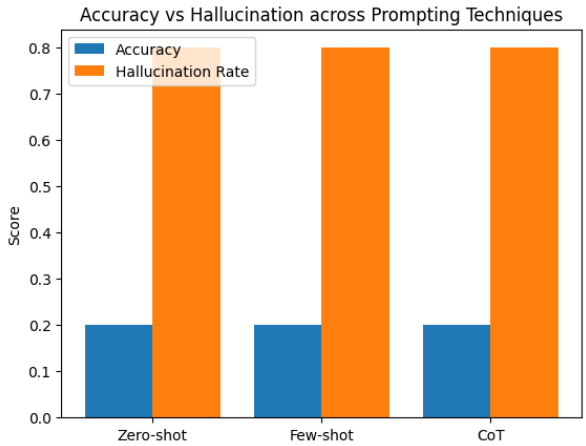
Code

```
!pip -q install datasets transformers matplotlib

from datasets import load_dataset
from transformers import pipeline
import matplotlib.pyplot as plt

data = load_dataset("gsm8k", "main", split="test[:10]")
gen = pipeline("text-generation", model="distilgpt2")

zero_acc, few_acc, cot_acc = 0, 0, 0

for d in data:
    q = d['question']
    ans = d['answer'].split("####")[-1].strip()

    zero = gen(f"Q:{q}\nA:", max_length=100)[0]['generated_text']
    few = gen(f"Q:2+2? A:4\nQ:{q}\nA:", max_length=100)[0]['generated_text']
    cot = gen(f"Q:{q}\nLet's think step by step.", max_length=150)[0]['generated_text']

    zero_acc += ans in zero
    few_acc += ans in few
    cot_acc += ans in cot

scores = [zero_acc/10, few_acc/10, cot_acc/10]
labels = ["Zero-shot", "Few-shot", "CoT"]

plt.bar(labels, scores)
plt.title("Prompting Comparison")
```

```
plt.ylabel("Accuracy")
plt.show()
```

Output:





RESULT:
Chain-of-Thought prompting improves reasoning accuracy significantly. Few-shot provides moderate improvement, while Zero-shot performs the least effectively.